Course Scheduling Problem – Project Report

0. Group members:

Vahhab Balandomanesh

Mohammadmahdi Kazemi

1. Introduction

The goal of this project is to solve the Course Scheduling Problem (CSP), where a student's course requests are assigned to

academic quarters while respecting various constraints. Each student provides:

• Registered quarters in which they can take courses.

• A list of course requests, where each request may have multiple course options.

• A list of courses already completed.

Each course contains:

• The quarters in which it is offered.

• Minimum and maximum number of units.

• A list of prerequisite courses that must be completed before enrollment.

The main task is to generate a feasible schedule that satisfies all constraints while maximizing the number of requests fulfilled.

2. Constraints

The project enforces the following rules:

1. Unique enrollment: Each course can be taken at most once.

2. Quarter availability: A course can only be scheduled in quarters in which it is offered and the student is registered.

3. Prerequisites: All prerequisite courses must be completed in earlier quarters or be already taken.

4. Unit limits: Each quarter must satisfy minimum and maximum unit limits based on the sum of the minimum units of

assigned courses.

5. Request options: Each request allows selecting at most one course from its list of options.

These constraints are carefully checked during schedule generation to ensure a valid solution.

3. Input Data

The project uses two input files:

1. courses.json: Contains all course information, including course ID, name, min/max units, prerequisites, and offered

quarters.

Example entry:

2. {

3. "CS221": {

4. "cid": "CS221",

5. "name": "Artificial Intelligence",

6. "minUnits": 3,

7. "maxUnits": 4,

8. "prereqs": ["CS109","CS161"],

9. "quarters": ["Aut","Win"]

10. }

11. }

12. profile.txt: Describes the student's profile, including registered quarters, unit limits, courses already taken, and course

requests.

Example entry:

13. minUnits 0

14. maxUnits 15

15. register Aut2015

16. register Win2016

17. taken CS106B

18. request CS229 or CS205A in Aut2015,Spr2016

19. request CS246 in Spr2016

4. Approach and Design

The problem is solved using a backtracking algorithm with constraint checking and heuristic optimization. The main steps are:

1. Data Loading:

o load_courses() reads courses.json into a dictionary mapping course IDs to their attributes.

o load_profile() parses profile.txt to extract registered quarters, taken courses, min/max units, and requests.

2. Slot Computation:

o For each course in the requests, the algorithm computes allowed slots, i.e., indices of registered quarters

where the course can be taken based on its offered quarters and optional request-specific constraints.

3. Backtracking Solver:

o The algorithm recursively assigns courses to quarters, ensuring constraints are satisfied.

o Constraints checked at each step:

▪ Prerequisites: Only assign if all required courses are completed before the target quarter.

▪ Unit limits: Prevent assignment if the quarter would exceed maxUnits.

▪ Requests: Only one course per request is assigned.

▪ 'After' constraints: Some requests may require a course to be scheduled after another.

o Heuristic optimization: Requests with fewer scheduling options are considered first to prune the search space

effectively.

4. Scoring and Selection:

o Among valid schedules, the solution maximizes:

▪ The number of fulfilled requests.

▪ Secondary criterion: the sum of request weights (if specified).

5. Output:

o If a valid schedule is found, the program outputs the chosen quarter for each course and total units per

quarter.

o If no valid schedule exists, a clear message is printed.

5. Challenges and Solutions

• Prerequisite dependencies across requests: Some courses depend on others that may also appear in future requests.

The algorithm tracks assigned courses and remaining possibilities to ensure all prerequisites can be satisfied.

• Unit limits per quarter: Ensuring minUnits and maxUnits are respected required updating total units dynamically during

backtracking and pruning infeasible paths early.

• Request options: Selecting only one course from each request while exploring multiple possible slots added complexity,

solved by recursive backtracking with careful state management.

6. Results

Example output for a sample profile:

Schedule:

CS229 -> Aut2015

CS246 -> Spr2016

Units per term:

Aut2015: 3

Win2016: 0

Spr2016: 3

Aut2016: 0

The algorithm successfully assigns courses to valid quarters while respecting all constraints and maximizing request fulfillment.

7. Conclusion

This project demonstrates a practical application of Constraint Satisfaction Problems in academic scheduling. Using backtracking

with heuristic ordering allows finding feasible schedules efficiently even when multiple constraints interact. The design ensures

correctness, flexibility for different profiles, and clear output for evaluation.